EthioChat: Master Build Prompt for Coding Agent

Paste everything below the line into your agent (Claude Code, Cursor, etc.). Build in the phases given; do not skip the acceptance gates.

---

0. ROLE AND MISSION

You are a principal mobile and backend engineer with 15+ years of experience shipping chat, fintech and super-app products at scale in low-bandwidth markets. Build **EthioChat**, a WeChat-class super-app for Ethiopia and the Horn of Africa: messaging, calls, social feed, channels, official accounts, mini programs and a mobile wallet in one simple app.

Non-negotiable principles

1. **Works on bad networks.** Ethiopia has frequent 2G/3G, packet loss and power cuts. Offline-first, resumable, tiny payloads.
2. **Works on cheap phones.** Target Android 8+ with 2 GB RAM; APK/AAB under 30 MB; cold start under 2.5 s on a low-end device.
3. **Simple.** A first-time user who has never used a super-app must send a message in under 30 seconds. Maximum 3 taps to any core action.
4. **Local first.** Amharic, Afaan Oromo, Tigrinya, Somali, English. Ethiopian calendar. ETB currency. +251 phone numbers. Telebirr, CBE Birr, Chapa.
5. **Correct and testable.** Every feature ships with tests. Regression suite must stay green.

Stack (fixed)

- Client: **Flutter (stable) + Dart 3**, null-safe, sound types
- State: **Riverpod 2** (code-gen) + **freezed** + **go_router**
- Local DB: **Drift (SQLite)** with FTS5 for search; **flutter_secure_storage** for keys
- Backend: **Supabase** (Postgres, Auth, Realtime, Storage, Edge Functions in Deno/TypeScript, pg_cron)
- Push: Firebase Cloud Messaging (data messages) plus local notifications
- Calls: **WebRTC** via LiveKit (self-hosted or cloud) with TURN over TCP/443 fallback
- Maps (later phase): MapLibre with OpenStreetMap tiles
- Crash and analytics: Sentry plus privacy-respecting event analytics

---

1. BRAND AND DESIGN SYSTEM

Palette (strict)

Token	Hex	Use
`goldDeep`	`#B8860B`	pressed states, borders
`goldShine`	`#D4AF37`	primary brand, buttons, active icons
`goldLight`	`#F4D77A`	highlights, gradient end
`cream`	`#FFF8E7`	light backgrounds, sent bubbles
`creamDark`	`#F3E9CC`	cards, dividers
`black`	`#0B0B0B`	dark background, primary text on light
`blackSoft`	`#1A1A1A`	dark surfaces, received bubbles in dark mode
`danger`	`#C0392B`	errors only

Rules

- Light theme: cream background, black text, gold accents. Dark theme: black background, cream text, gold accents.
- "Shiny gold" = `LinearGradient(topLeft→bottomRight, [#B8860B, #D4AF37, #F4D77A, #D4AF37])` used on the primary button, logo, wallet card and selected tab indicator only. Do not overuse; gold on every element destroys hierarchy.
- Contrast: all text meets WCAG AA (4.5:1). Gold text on cream is forbidden; use black text on gold.
- Font: **Noto Sans Ethiopic** for Ge'ez script plus **Inter** for Latin; bundle subsets to keep size small.
- Spacing grid 4/8/16/24; corner radius 14; touch targets at least 48 dp; minimum body text 16 sp; respect system font scaling to 200%.
- Bottom navigation, exactly 4 tabs: **Chats · Contacts · Discover · Me**. Wallet sits in Me and Discover.
- Icons: one outline icon set, 2 px stroke. Empty states have a short friendly line plus one primary action.

Starter theme code (implement and extend)

```
import 'package:flutter/material.dart';

class EthioColors {
  static const goldDeep = Color(0xFFB8860B);
```

```
  static const gold = Color(0xFFD4AF37);
  static const goldLight = Color(0xFFF4D77A);
  static const cream = Color(0xFFFFF8E7);
  static const creamDark = Color(0xFFF3E9CC);
  static const black = Color(0xFF0B0B0B);
  static const blackSoft = Color(0xFF1A1A1A);
  static const danger = Color(0xFFC0392B);
  static const goldGradient = LinearGradient(
    begin: Alignment.topLeft,
    end: Alignment.bottomRight,
    colors: [goldDeep, gold, goldLight, gold],
  );
}

ThemeData buildTheme(Brightness b) {
  final dark = b == Brightness.dark;
  final scheme = ColorScheme(
    brightness: b,
    primary: EthioColors.gold,
    onPrimary: EthioColors.black,
    secondary: EthioColors.goldDeep,
    onSecondary: EthioColors.cream,
    surface: dark ? EthioColors.blackSoft : EthioColors.cream,
    onSurface: dark ? EthioColors.cream : EthioColors.black,
    error: EthioColors.danger,
    onError: Colors.white,
  );
  return ThemeData(
    useMaterial3: true,
    colorScheme: scheme,
    scaffoldBackgroundColor: dark ? EthioColors.black : EthioColors.cream,
    fontFamily: 'Inter',
    fontFamilyFallback: const ['NotoSansEthiopic'],
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
        minimumSize: const Size.fromHeight(52),
        shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(14)),
      ),
    ),
  );
}
```

2. ARCHITECTURE

Clean architecture, feature-first

```
lib/
  core/        (theme, l10n, network, db, errors, utils, crypto, logging)
  features/
    auth/ chats/ contacts/ calls/ moments/ channels/
    official_accounts/ mini_programs/ wallet/ discover/ profile/ settings/
    (each: data/ domain/ presentation/)
  app.dart  main.dart
supabase/
  migrations/  functions/  seed.sql  tests/
```

- Domain layer is pure Dart (no Flutter, no Supabase imports). Repositories are interfaces; Supabase and Drift are implementations.
- All errors are typed (`sealed class Failure`). No raw exceptions reach the UI.
- Single source of truth for UI = **local Drift database**. Network only syncs into it. Widgets watch Drift streams.
- Feature flags table (remote config) to disable any feature server-side without a release.

---

3. SUPABASE DATABASE

Create versioned SQL migrations. Use `uuid` PKs (`gen_random_uuid()`), `timestamptz` , soft deletes (`deleted_at`) and RLS **enabled on every table**.

Core tables

- `profiles(id → auth.users, ethio_id unique, display_name, avatar_url, bio, language, phone_hash, last_seen_at, privacy jsonb)`
- `devices(id, user_id, fcm_token, platform, identity_pubkey, prekeys jsonb, last_active)`
- `contacts(owner_id, contact_id, alias, status[pending|accepted|blocked], created_at)`
- `conversations(id, type[direct|group|channel], title, avatar_url, created_by, last_message_id, last_message_at)`
- `conversation_members(conversation_id, user_id, role[owner|admin|member], muted_until, last_read_seq, joined_at)`

- `messages(id uuid client-generated, conversation_id, sender_id, seq bigint server-assigned, type[text|image|voice|video|file|location|contact|sticker|payment|redpacket|system], body text/ciphertext, media jsonb, reply_to, edited_at, deleted_at, created_at default now())` with unique `(conversation_id, seq)`
- `message_receipts(message_id, user_id, delivered_at, read_at)`
- `moments(id, author_id, text, media jsonb, visibility, created_at)`, `moment_likes`, `moment_comments`
- `official_accounts(id, owner_id, name, verified, category)`, `oa_posts`, `oa_subscriptions`
- `mini_programs(id, name, icon, manifest_url, version, publisher_id, status, permissions jsonb)`
- `wallets(user_id, currency default 'ETB', status)`, **`ledger_entries`** (double-entry: `id, txn_id, account, direction[debit|credit], amount_minor bigint, currency, created_at`; immutable, no UPDATE or DELETE grants)
- `transactions(id, type[topup|p2p|redpacket|merchant|withdraw|bill], status[pending|success|failed|reversed], amount_minor, fee_minor, provider, provider_ref, idempotency_key unique, meta jsonb)`
- `redpackets(id, sender_id, conversation_id, total_minor, count, mode[random|equal], expires_at)`, `redpacket_claims`
- `reports`, `blocks`, `audit_log`, `feature_flags`

Required DB logic

- **Per-conversation sequence:** trigger `assign_message_seq()` using `SELECT ... FOR UPDATE` on a counter row in `conversations` so `seq` is gapless and monotonic. Clients order by `seq`; pending local messages sort after the last known seq.
- **Idempotent insert:** client-generated message UUID plus `ON CONFLICT (id) DO NOTHING` so retries never duplicate.
- **RLS examples:** a user can `SELECT` messages only where they are a member of the conversation; `INSERT` only where `sender_id = auth.uid()` and they are a member; `ledger_entries` writable only by the service role (Edge Functions).
- Indexes: `(conversation_id, seq desc)`, `(user_id, last_read_seq)`, GIN on search fields, partial index on unread.
- `pg_cron` jobs: expire red packets and refund, purge soft-deleted media after 30 days, rebuild daily stats.
- **Storage buckets:** `avatars` (public), `media` (private, signed URLs, 15-min expiry), `mini-programs` (public, versioned). Enforce size limits: image 10 MB, voice 5 MB, video 50 MB, file 100 MB.

- Write pgTAP tests for every RLS policy (positive and negative cases).

---

4. CORE ALGORITHMS AND WORKFLOWS

4.1 Authentication

1. Phone number input with +251 default and validation (`^\+251(9|7)\d{8}$`). Also accept 09xxxxxxxx and normalise.
2. Supabase phone OTP (SMS via a local-friendly provider, with a **Telegram or voice-call OTP fallback** since SMS delivery is unreliable). Rate limit: 3 codes per 10 min per number and per device.
3. First login: pick display name, avatar (optional), language. Generate device keypair. Register FCM token.
4. Optional app lock (biometrics or PIN). Session persists; refresh token handled silently.
5. Contact discovery: hash phone numbers (SHA-256 with server pepper) on-device; match through an Edge Function; never upload raw address books.

4.2 Offline-first messaging (the heart of the app)

Send pipeline (outbox pattern)

1. User taps send → create message with client UUID, status `pending` , write to Drift **immediately**; UI shows it with a clock icon.
2. `OutboxWorker` (single isolate, FIFO per conversation) picks up pending rows. Upload media first (resumable, chunked 256 KB, retry from last chunk), then insert the message row.
3. Exponential backoff with jitter: 1 s, 2 s, 4 s … capped at 60 s; resets on connectivity change.
4. Success → status `sent` (one tick). Realtime receipt → `delivered` (two ticks). Read → two gold ticks.
5. After 10 failures show "Tap to retry"; never silently drop.

Receive and sync

- Realtime channel per user subscribes to member conversations (Postgres changes filtered by membership; batch events at 100 ms to cut re-renders).
- On reconnect or app resume run **delta sync**: for each conversation request `messages WHERE seq > local_max_seq ORDER BY seq LIMIT 100` , loop until empty. Fetch newest conversations first.
- Gap detection: if received `seq > local_max_seq + 1` , fetch the gap before displaying.

- FCM data push wakes the app; the payload contains only conversation ID and seq (no plaintext); the app syncs and creates the local notification.
- Pagination: infinite scroll upward, 30 messages per page; use `ListView` with `reverse: true` and stable keys.

Presence, typing and receipts

- Use Supabase Realtime **Presence** and **Broadcast** (ephemeral, not stored). Typing events throttled to one per 3 s; presence heartbeat every 30 s; cease on background.
- Read receipts are batched: send one `last_read_seq` update per conversation, debounced 1 s.

Other chat features Reply, forward, edit within 15 min, delete for me or everyone (24 h), mentions, voice notes (Opus, waveform, hold-to-record, slide-to-cancel, lock), image sending with on-device compression (max 1280 px, JPEG/WebP quality 70), thumbnails and blurhash placeholders, video compression, file sharing, location sharing, stickers, message search (Drift FTS5 with Ge'ez-aware tokenizer plus case and diacritic folding), pinned chats, archive, mute, drafts persisted per chat, disappearing messages timer, group chats up to 500 members with roles, QR code join, invite links.

4.3 End-to-end encryption

- **Phase 1:** TLS plus encryption at rest; clearly state this in privacy settings.
- **Phase 2 (mandatory design now, implement after MVP):** 1:1 chats use X3DH key agreement plus Double Ratchet (X25519, AES-256-GCM, HKDF) via a vetted library; never invent crypto. Keys live in secure storage; devices table holds only public prekeys. Group chats use sender keys. Provide safety-number verification screen.
- Design `messages.body` so it can hold ciphertext without schema change.

4.4 Calls

- 1:1 and group voice and video through LiveKit. Signalling through Supabase Realtime; ringing through high-priority FCM plus full-screen intent (CallKit/ConnectionService via `flutter_callkit_incoming`).
- Adaptive bitrate; **audio-only auto-fallback** when bandwidth below 60 kbps; Opus with DTX and FEC; TURN on TCP 443.
- Call log stored as system messages.

4.5 Moments, Channels and Official Accounts

- Moments: chronological feed of friends only; visibility (all, selected, private); likes and comments visible only to mutual friends; image grid up to 9; lazy image loading; feed pagination with keyset (`created_at, id`).

- Channels: short video (max 60 s), vertical swipe, preload next 2 clips, data-saver mode defaults to 360p on mobile data, upload resumable.
- Official Accounts: verified publishers (churches, banks, media, government offices, shops) with subscription feed, article cards, in-chat menu and auto-replies; admin web console (Phase 4).

4.6 Mini Programs

- Each mini program is a signed bundle (HTML/JS/CSS) hosted in the `mini-programs` bucket with a `manifest.json` (id, version, permissions, entry, hash).
- Run in a sandboxed `WebView` with a **JS bridge** exposing only whitelisted, permission-gated APIs: `getUserInfo`, `pay`, `scan`, `share`, `getLocation`, `storage`. Verify the bundle hash and signature before load; cache for offline.
- Seed 3 first-party mini programs: **Bills** (electricity, water, telecom), **Airtime/Data top-up**, **Local Marketplace**.

4.7 Wallet and payments (highest risk; build with maximum rigor)

- All money movement happens **only in Edge Functions** with the service role, never from the client.
- **Double-entry ledger:** every transaction produces balanced debit and credit entries inside one DB transaction; balance = sum of entries; never store a mutable balance as the source of truth.
- **Idempotency:** every request carries `Idempotency-Key`; replay returns the original result.
- **Top-up and withdraw:** integrate Telebirr, CBE Birr and Chapa behind a `PaymentProvider` interface (adapter pattern) with webhooks verified by signature and replay protection. Provider credentials in Supabase secrets only.
- **P2P transfer:** PIN confirmation (6-digit, hashed with Argon2, 5 attempts then 30-min lock), daily and per-transaction limits, tiered KYC (Tier 0 phone only, Tier 1 Fayda/National ID, Tier 2 full), velocity and anomaly rules, instant receipt message in chat.
- **QR payments:** merchant QR is a signed payload; scanning shows merchant name and amount for confirmation.
- **Red packets:** random split algorithm (double-mean method with a minimum of 1 birr), atomic claim with `SELECT ... FOR UPDATE SKIP LOCKED`, expiry refund job.
- **Reconciliation job** nightly: compare provider reports to ledger; alert on mismatch.
- Comply with National Bank of Ethiopia directives; include terms, consent and receipts. Flag clearly in docs where legal licensing is required before launch.

---

5. CONNECTIVITY AND PERFORMANCE

- `ConnectivityService` combines `connectivity_plus` with an actual reachability ping (Supabase health endpoint) and exposes `online | weak | offline`; show a slim banner ("Connecting…", "Waiting for network").
- **Data saver mode** (default on for mobile data): thumbnails only, no video autoplay, lower-quality uploads.
- Payloads: use `select` with explicit columns, gzip, compact JSON; never `select *`.
- Request timeouts 15 s; retries only for idempotent calls; circuit breaker after 5 consecutive failures.
- Images: `cached_network_image` with disk cache cap 200 MB (LRU); blurhash placeholders.
- Budget: 60 fps scrolling in chat lists with 10,000 messages; memory under 250 MB; app idle battery use below 2% per hour; startup path lazy-loads non-critical features.
- Use `compute`/isolates for compression, crypto and JSON parsing of large batches.
- Android: foreground service only for calls; WorkManager for background outbox and sync; handle OEM battery killers (Tecno, Infinix, Xiaomi) with an in-app guide.

---

6. SCREENS (build all, with loading, empty, error and offline states)

1. Splash and language picker
2. Phone entry → OTP → profile setup
3. **Chats list** (search bar, unread badges, pinned, swipe actions, floating compose)
4. **Chat screen** (gold-tick receipts, reply swipe, long-press menu, attachment sheet, voice recorder, scroll-to-bottom with unread count)
5. New chat / create group / group info and admin tools
6. **Contacts** (A–Z index, requests, blocked, search by Ethio ID / phone / QR)
7. **Discover** (Moments, Channels, Official Accounts, Mini Programs, Scan, Nearby)
8. Moments feed and composer; Channels player
9. Official account page and article reader
10. Mini program container with capsule menu (close, share, refresh)
11. Call screens: incoming, outgoing, in-call (mute, speaker, camera flip, add member)
12. **Wallet:** balance card (gold gradient), top-up, send, receive/QR, scan to pay, bills, history, receipt, PIN setup
13. **Me:** profile, my QR, settings

14. Settings: account security, privacy (who can find me, last seen), notifications, chat backup, storage manager, data saver, language, theme, blocked users, help, about
15. Report and block flows; onboarding tips

UX rules: one primary action per screen; no more than 5 items in any menu; undo snackbars instead of confirmation dialogs where safe; haptics on send; skeleton loaders (cream shimmer); fully accessible with TalkBack labels.

---

7. LOCALIZATION

- ARB files for `en`, `am` (Amharic), `om` (Afaan Oromo), `ti` (Tigrinya), `so` (Somali). No hardcoded strings. Pluralization rules per language.
- **Ethiopian calendar** support in date display and pickers (toggle in settings); Ethiopian 12-hour clock option ("ሰዓት" system) for time display.
- Amharic keyboard friendly: ensure input fields don't break Ge'ez composition; test emoji and Ge'ez mixed text, RTL-safe layouts (future Arabic).
- Currency formatting `ETB 1,250.50`; phone formatting `+251 9X XXX XXXX`.

---

8. SECURITY AND PRIVACY

- RLS everywhere, least-privilege service roles, secrets never in the client or repo. Only the anon key ships in the app.
- Certificate pinning for the API domain; root and jailbreak detection (warn only); screenshot block on wallet and PIN screens (`FLAG_SECURE`).
- Rate limiting and abuse control in Edge Functions; spam and scam heuristics (link warnings, new-account forwarding limits).
- OWASP MASVS L1 checklist; dependency audit in CI; obfuscated release build (`--obfuscate --split-debug-info`).
- Privacy: minimal data, account deletion flow, data export, clear consent; privacy policy and terms in all supported languages.
- Content moderation: report queue, block lists, admin tools; comply with local law.

---

9. REGRESSION AND QUALITY STRATEGY

The agent must treat tests as part of the deliverable.

- **Unit tests** (target ≥ 85% on domain and data layers): outbox state machine, backoff, seq gap detection, red-packet split (sum always equals total, min amount respected, 10,000 randomized runs), ledger balance invariants, phone normalisation, Ethiopian calendar conversion, hash-matching.
- **Widget and golden tests** for every screen in light and dark, at 1.0× and 2.0× font scale, in `en` and `am`.
- **Integration tests** (`integration_test`) for critical journeys: signup → send message → offline send → reconnect delivery; group create; media send; P2P transfer; red packet claim; call setup.
- **Backend tests:** pgTAP for RLS and triggers; Deno tests for Edge Functions; webhook replay and duplicate-idempotency tests.
- **Network chaos tests:** simulate 2G latency, 30% packet loss, mid-upload disconnect, airplane-mode toggling, app kill during send; assert zero duplicate or lost messages.
- **Concurrency tests:** 50 clients sending to one conversation, with seq strictly increasing and gapless; 100 simultaneous red-packet claims never over-allocate.
- **Regression gate in CI (GitHub Actions):** format, `dart analyze --fatal-infos`, unit, widget, golden, integration on emulator, pgTAP, Edge Function tests, APK size check (fail above 30 MB), performance smoke test. No merge unless all green.
- Feature changes require a test added first for any bug fixed (bug → failing test → fix).
- Crash-free sessions target ≥ 99.5%.

---

10. DELIVERY PHASES AND ACCEPTANCE GATES

Phase	Scope	Gate
1 MVP	Auth, profile, contacts, 1:1 + group text chat, offline outbox, push, media, theming, localization (en, am)	Chaos tests pass; 0 lost messages; startup < 2.5 s
2	Voice notes, voice/video calls, search, E2EE for 1:1, Oromo/Tigrinya/Somali	Call setup < 4 s on 3G; crypto review
3	Wallet: top-up, P2P, QR pay, red packets, bills	Ledger invariants proven; provider sandbox end-to-end; PIN security tests
4	Moments, Channels, Official Accounts + admin console	Feed smooth at 60 fps; moderation tools live
5	Mini programs platform, marketplace, nearby, growth features	Sandbox security review

For each phase, the agent must: (1) restate the plan briefly, (2) write migrations and tests first, (3) implement, (4) run the full regression suite, (5) report what was built, test results, known gaps and next step. Never claim something works without running it. If a requirement is ambiguous, choose the simplest safe option, document it in `DECISIONS.md` and continue.

---

11. REPOSITORY DELIVERABLES

`README.md` (setup in 10 minutes), `ARCHITECTURE.md`, `DECISIONS.md`, `SECURITY.md`, `.env.example`, Supabase migration files plus seed data, `Makefile` or scripts (`make dev`, `make test`, `make release`), Fastlane/Play Store release pipeline, ER diagram, API docs for Edge Functions, a QA checklist for manual testing on real low-end devices, and a launch checklist covering licensing, privacy policy, store listing and support channels.

---

12. START NOW

Begin with Phase 1. First output: (a) the folder structure, (b) the complete SQL migration for Phase 1 tables with RLS and the seq trigger, (c) the Drift schema, (d) the theme and the `OutboxWorker` with its unit tests. Then continue until the Phase 1 gate passes. Ask me only for credentials (Supabase URL and keys, FCM, SMS provider, payment sandbox) when you truly need them.